

Unit- 5
Chaper 7

Architecture of ARM Cortex-M



Introduction

- Later ARM 7 Was Followed By ARM 9 And ARM 11
- Means More Features Were Added To It, But The Basic Architecture Was Similar To ARM7
- The Latest Series Of ARM 7 Processors are Designated as Cortex
- They Have Slightly Different Architecture As Compared to arm processor.

ARM Processor Families

➤ Cortex- A Series (Application)

- High Performance Processors Capable of Full Operating System (OS) Support;
- Applications Include Smartphones, Digital TV, Smart Books, Home Gateways Etc.

➤ Cortex- R Series (Real-time)

- High Performance for Realtime Applications;
- High Reliability
- Applications Include Automotive Braking System, Powertrains Etc.



➤ Cortex-M Series (Microcontroller)

- Cost-sensitive Solutions For Deterministic Microcontroller Applications;

- Applications Include Microcontrollers, Mixed Signal Devices, Smart Sensors, Automotive Body Electronics And Airbags; More Recently IOT.

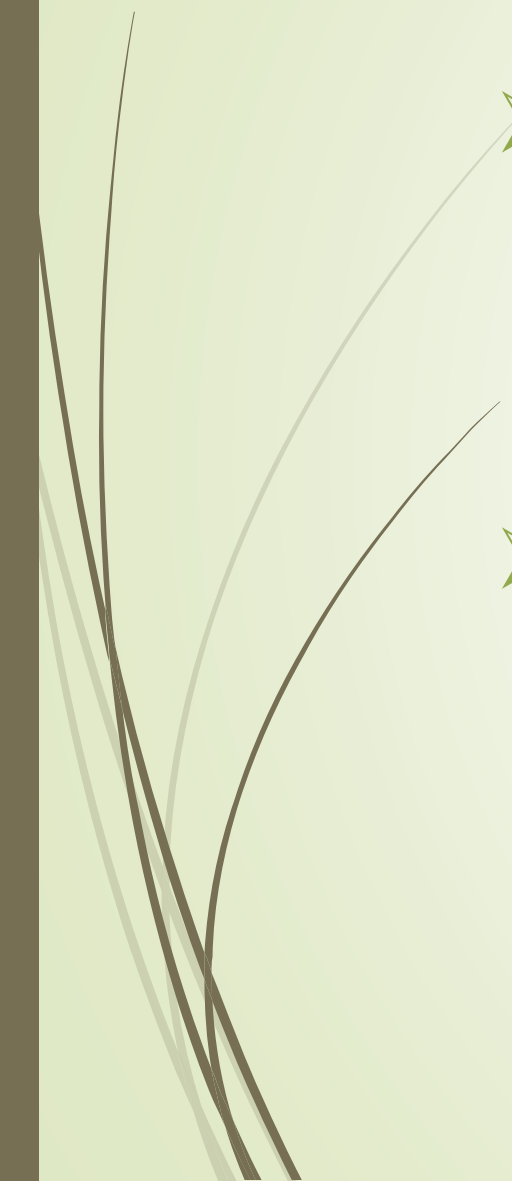
➤ Secure core Series

- High Security Applications

➤ Previous Classic Processors: Include ARM7, ARM9, ARM11 Families

7.1 Cortex – M Processor

- Lets Begin Our Study On Cortex-m Processor Family.
- The Cortex-m Family has been Designed For Embedded Applications With The Use Of Microcontrollers.
- Cortex-m Family Is A Low End Of The Cortex Family
- But It Posses A Very Good Computational Capabilities.
- The M Series Has Many Variants Which Are M0, M0+, M1, M3, M4 And M7.
- Each Variant Has A Special Feature And Depending On The Application, The User Can Select A Particular Variant.

- 
- 
- one of the point in focus is low power capability and that feature is taken into consideration at every stage in processor design.
 - Lets see the brief reference to each of the members of the cortex-m family.

Cortex-M Processors Family

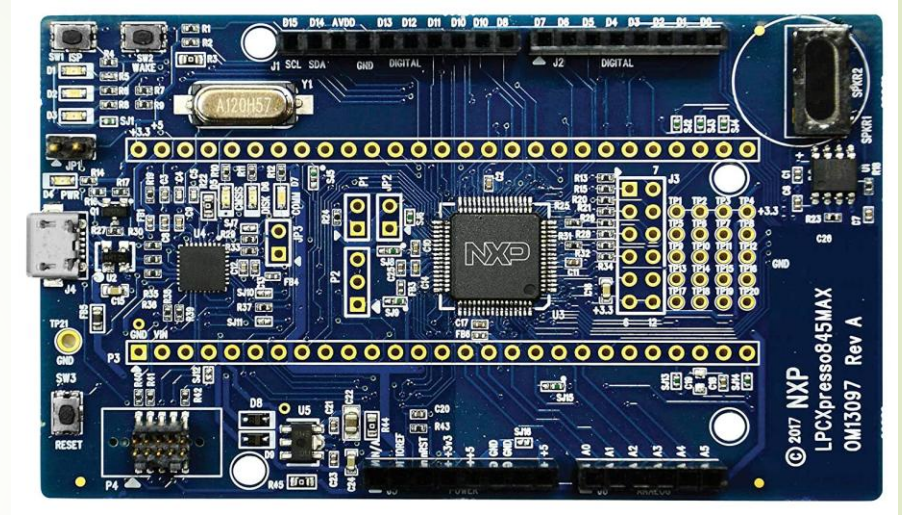
- **Cortex- M0:** It is very small processor (in size) on account of its very low gate count. It is meant to be used in very low power embedded applications.



- The Arm Cortex-M0 processor is one of the smallest Arm processors available.

- 
- 
- The Cortex-M0 has an exceptionally small silicon area, low power and minimal code footprint, enabling developers to achieve 32-bit performance at an 8-bit price point, bypassing the step to 16-bit devices.
 - The ultra-low gate count of the processor enables its deployment in analog and mixed signal devices.

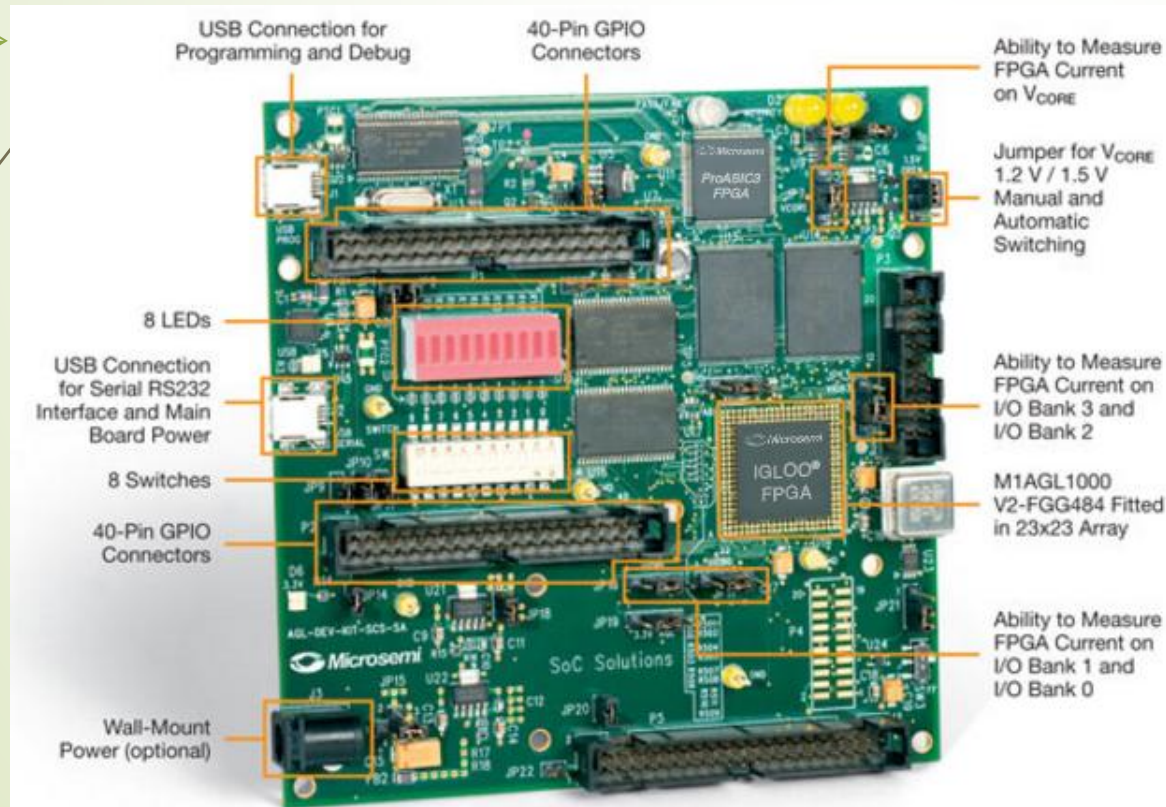
- **Cortex- M0+:** It is similar to M0, but can boast of lower power, a shorter pipeline and an additional feature termed 'Single Cycle execution' for I/O



- 32-bit, Low-Power Processor at an 8-bit Cost
- The Cortex-M0+ processor has the smallest footprint and lowest power requirements of all the Cortex-M processors. The low-power processor is

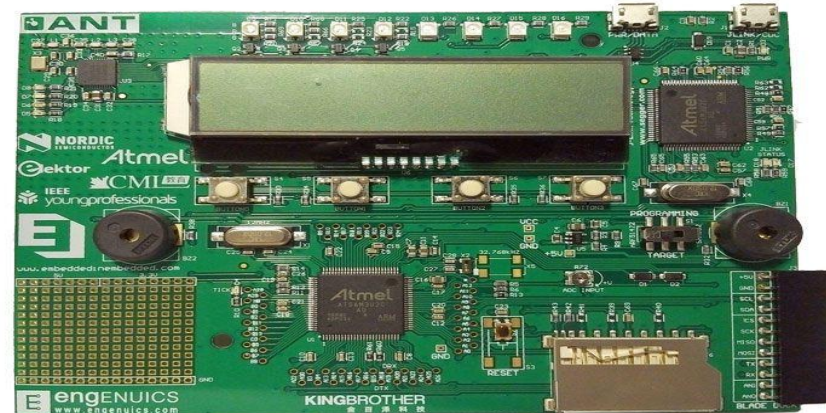
➤ **Cortex- M1:** It is optimized for FPGA designs and has a 'tightly coupled memory'.

➤ Implementation utilizing the memory blocks on the FPGA



Cortex- M0

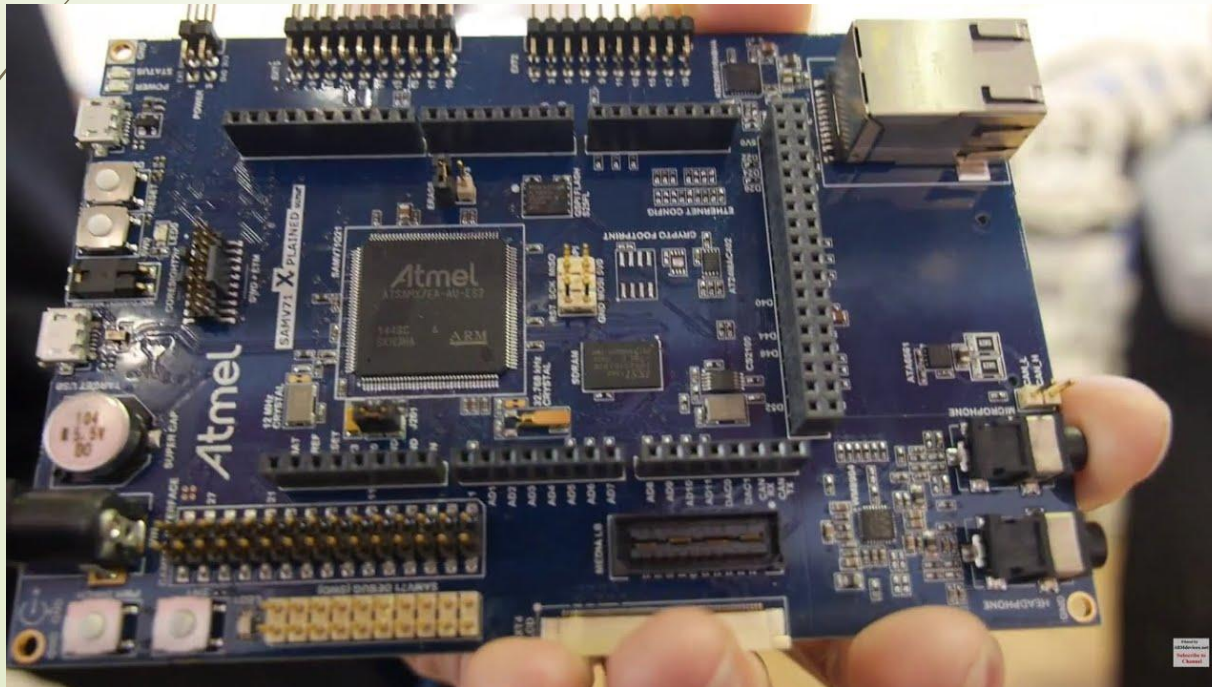
- **Cortex- M3:** This is more powerful processor and has features meant to handle more complex designs.
- It has a hardware divider which is not normally available in ARM processors.
- It also has a Multiply and Accumulator unit, which helps in fast DSP computations



- **Cortex- M4:** This is similar to M3, but has more features that cater to faster digital signal processing.



- **Cortex- M7:** It is the highest performing processor in the M series.
- It has advanced features like superscalar capability, longer pipeline, superior bus interfaces, rich DSP feature etc.

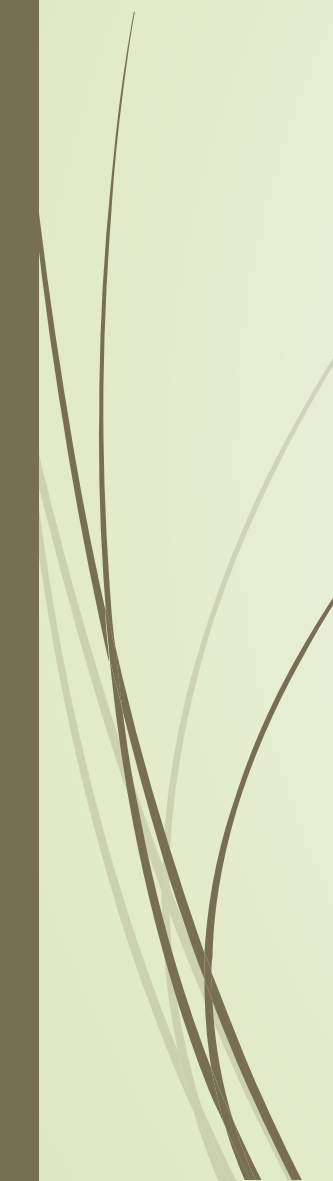


- 
- 
- Cortex M0, M0+ and M1 processors use the ARM v6-M ISA
 - Cortex M3, M4 and M7 processors use the ARM v7-M ISA

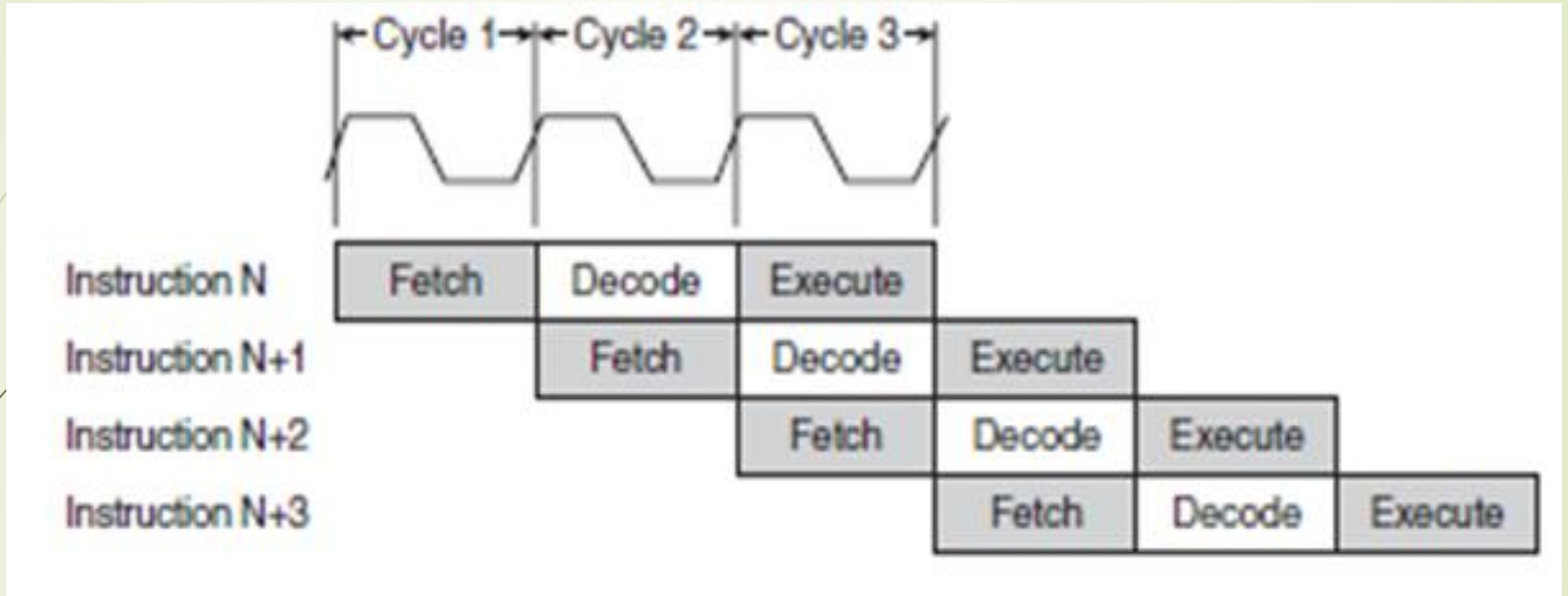


7.2 Cortex- M0

- ❑ Cortex-M0 was released in the year 2009.
- ❑ It was the smaller ARM processor with highly energy efficient
- ❑ It was good enough in performance for most embedded applications
- ❑ Based on its market review and performance, it was stated as 'Fastest ever licenced ARM processor'.

- 
- 
- ❑ It mainly focus on low power and small silicon area while keeping the performance high.
 - ❑ For this 32-bit MCU, the power silicon area are almost the same as that of 8 or 16 bit MCU.
 - ❑ Its performance is 2 to 10 times better than many 8 and 16 bit processors.

Three-stage pipeline for Cortex-M0



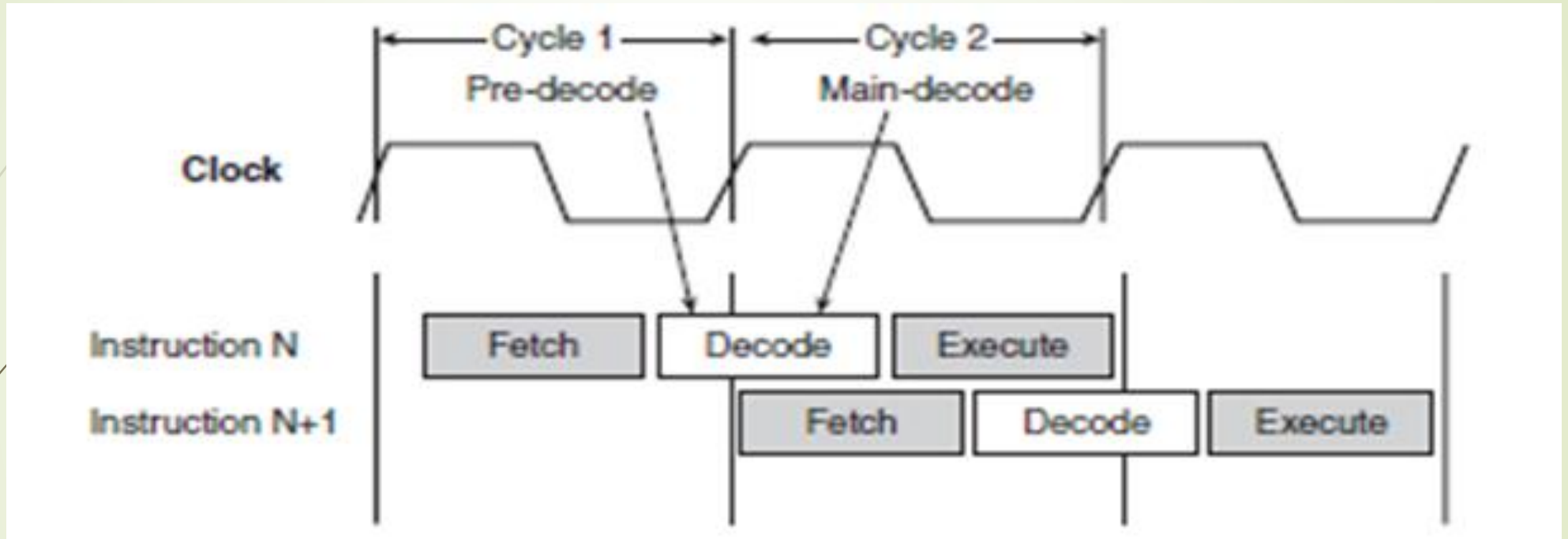
- ❑ The three stage pipeline of M0 is a regular fetch-decode-execute pipeline
- ❑ It takes 3 cycles to process an instruction

Wat about Cortex-M0+?

- ❑ Cortex-M0+ was released three years later.
- ❑ Performance wise, the core is an upgraded version of M0
- ❑ Uses the same software tools.
- ❑ But difference, M0 has 3-stage pipeline whereas M0+ has only 2-stages.
- ❑ The smaller pipeline helps in more power savings.
- ❑ Cortex-M0+ is ideal for academic purposes, boards development and also quite

- 
- ❑ The architectural features of all members of the Cortex-M series have many things in common
 - ❑ So, need to understand thorough concept of Cortex-M0 is the first step.
 - ❑ So we discuss here only on M0 architecture keeping in mind that the M0+ is almost same.
 - ❑ This makes it easier to understand other members of series.

Two-stage pipeline for Cortex-M0+



- ❑ M0+ has 2 stages
- ❑ fetch, decode & execute are completed in two cycles

Advantages for a designer to choose M0 for his design

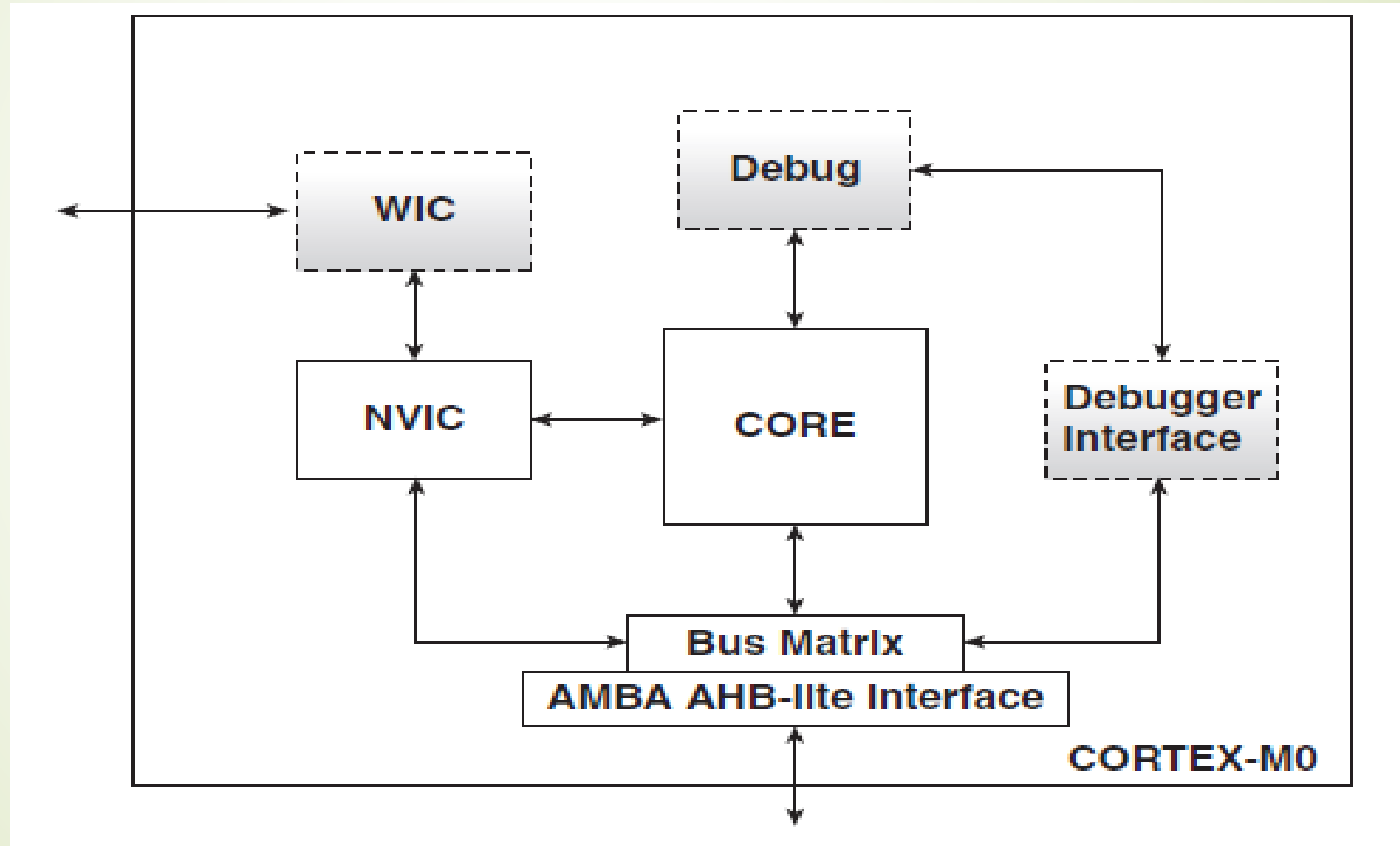
- ❑ Its architecture is simple and easy to learn and understand
- ❑ It has low gate count
- ❑ Its performance is quite good as far as comparison capability is concerned
- ❑ It operates at very low power levels and is energy efficient
- ❑ Its code density is high
- ❑ Its interrupt handling is deterministic
- ❑ It is upward compatible with the whole Cortex- M family
- ❑ It has an integrated Memory Protection Unit (Optional) and thus is capable of providing platform security.

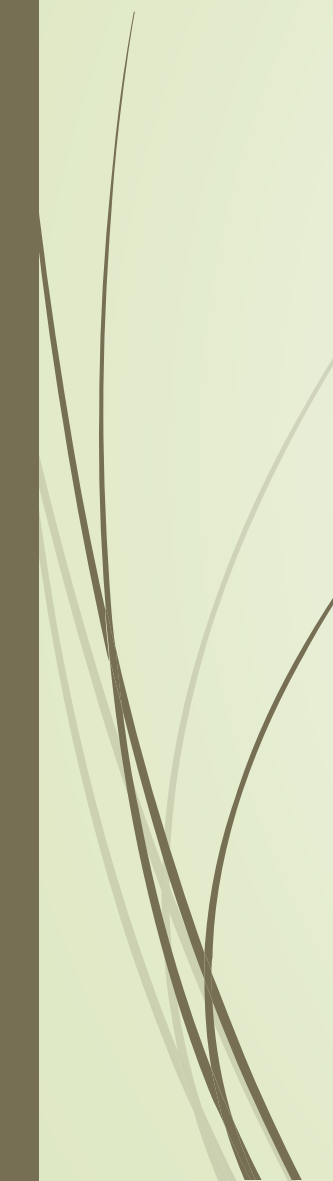
Features of Cortex-M0

- ❑ It is a 32-Bit processor using ARM v6M ISA.
- ❑ Most of its instructions have single cycle execution capability
- ❑ It has only 56 base instructions, through some instructions have more than one form of usage
- ❑ It is a three-stage pipeline design
- ❑ It is a Von Neuman design

7.2.2 The Core

Fig: Block diagram of Cortex-M0



- 
- 
- ☐ The components of the Cortex-M0 processor is show
 - ☐ The dotted components are optional
 - ☐ Mandatory components are the core, Nested Vectored Interrupt Controller (NVIC) and bus matrix.
 - ☐ The bus matrix that interfaces to the AMBA AHB-lite bus
 - ☐ The processor core is the computing engine which contains the registers and ALU
 - ☐ The instruction pipeline has 3 stages for M0 and 2 stages for M0+.

7.2.3 Nested Vectored Interrupt Controller (NVIC)

- ❑ This is the hardware block that handles all the interrupts that comes to the processor.
- ❑ Mechanism is found, that handles multiple interrupts and accept one of them on priority resolution schemes
- ❑ Once the specified interrupt is acknowledged, the interrupting device can access its interrupt handler
- ❑ The NVIC has been designed for interrupt latency to be low and deterministic.

7.2.4 AMBA AHB-lite

- ❑ AHB bus is a part of AMBA standard
- ❑ It is on-chip internal bus protocol for data transfers within the chip

7.2.5 Debug Unit

- ❑ It has a breakpoint and watch point units
- ❑ User can halt execution at points he has chosen and do active debugging
- ❑ The debugger access port is the interface by which the processor is connected to the host system
- ❑ debugging may be done by the JTAG interface

7.2.6 Wakeup interrupt controller

- ❑ It is optional unit which provides ultra low power sleep support
- ❑ As the processor is lowered down, most of its functional units including the NVIC are inactive
- ❑ In this state, if an interrupt arrives, it is handled by the wake up interrupt controller until the processor is brought back to active state.
- ❑ Thereafter, interrupt handling is done by NVIC

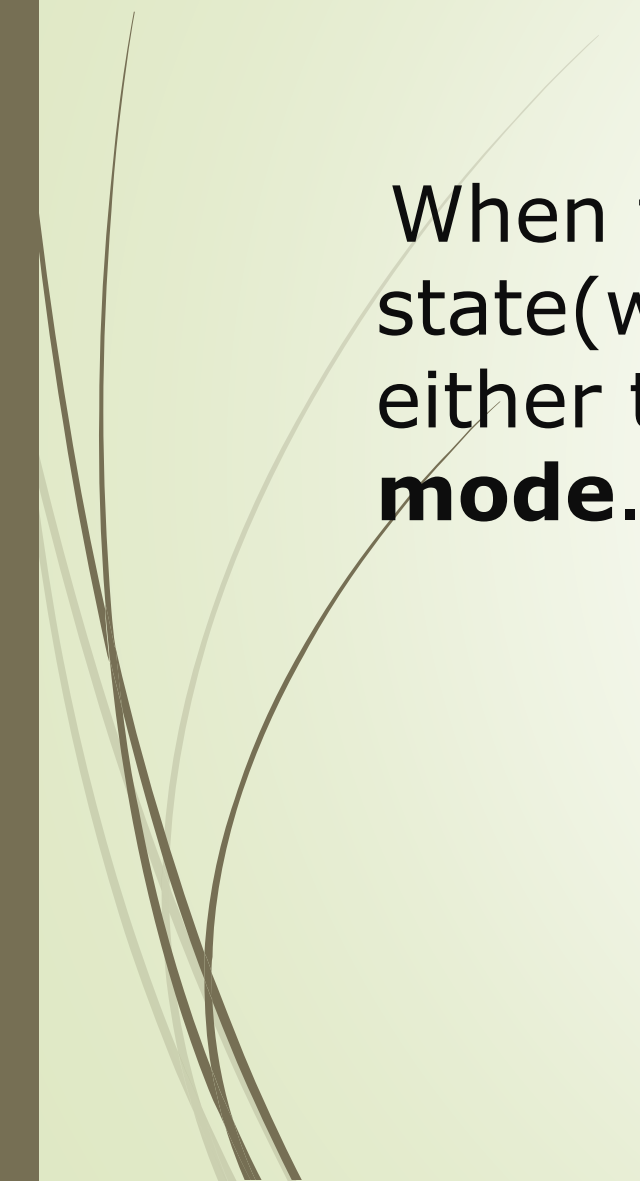
7.3 Modes and states

- ❑ It has Various modes and States for operation
 1. Processor States
 2. Thread Mode
 3. Handler Mode

7.3.1 Processor States

- ❑ Processor has 2 states
- ❑ When it is executing a program, it is said to be in the **Thumb State**.
- ❑ The other state is the **Debug state** .

7.3.2 Modes of operation

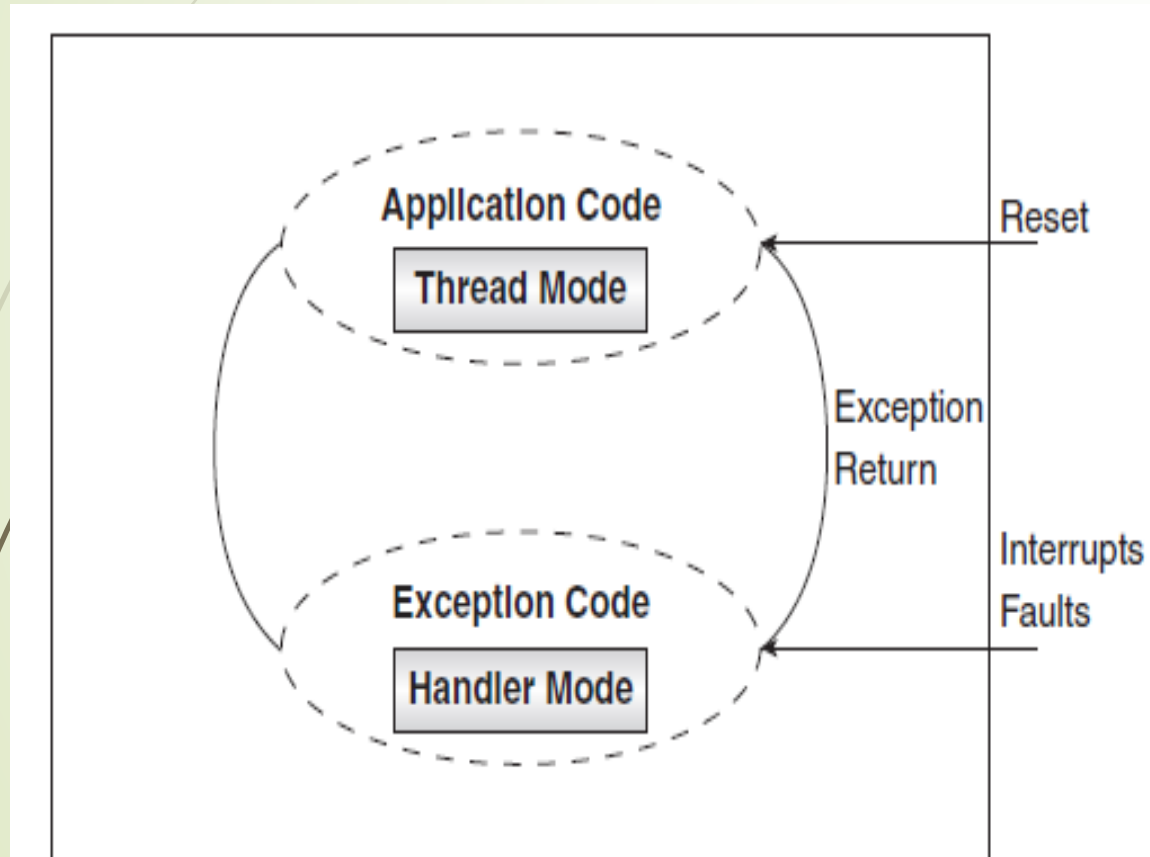


When the processor is in the Thumb state(when running a program), it can be either the **Thread mode** or the **Handler mode**.

7.3.2.1 Thread Mode

- ❑ When the processor is powered on, it is in the Thread Mode
- ❑ This is the mode, as it is entered on reset and also on return from an interrupt
- ❑ There are two stacks defined for the processor:
 - 1. Main Stack**
 - 2. Process Stack**
- ❑ In the Thread mode, either of them may be used and a control register can be configured to decide whether the processor should use the main stack or the process stack.

7.3.2.2 Handler mode



- This is the mode which is entered as a result of an interrupt
- figure shows the usage of these modes
- It is the **main stack**, that is used in this mode

Table 7.1 Difference between thread and handler modes

Processor mode	Use to execute	Stack used
Thread	Applications	Main Stack or Processor stack
Handler	Exception Handlers	Main Stack

If the processor has an OS running in it, then the thread mode will pertain to user tasks and the handler mode will be allocated for OS tasks as well as for interrupt handling

7.4 Programming model

- In an Assembly Language or a Compiler design programming, the programmer sees the processor as,
 - A set of Registers
 - A set of Instructions

7.4.1 Register Set

- Figure shows the registers of the core
- General purpose registers are all 32 bit and are named R0 to R12
- They are used for temporary data storage
- For holding operands during computations
- Many instructions can use only registers R0 to R7 and are called low registers
- There are other instructions where all registers can use

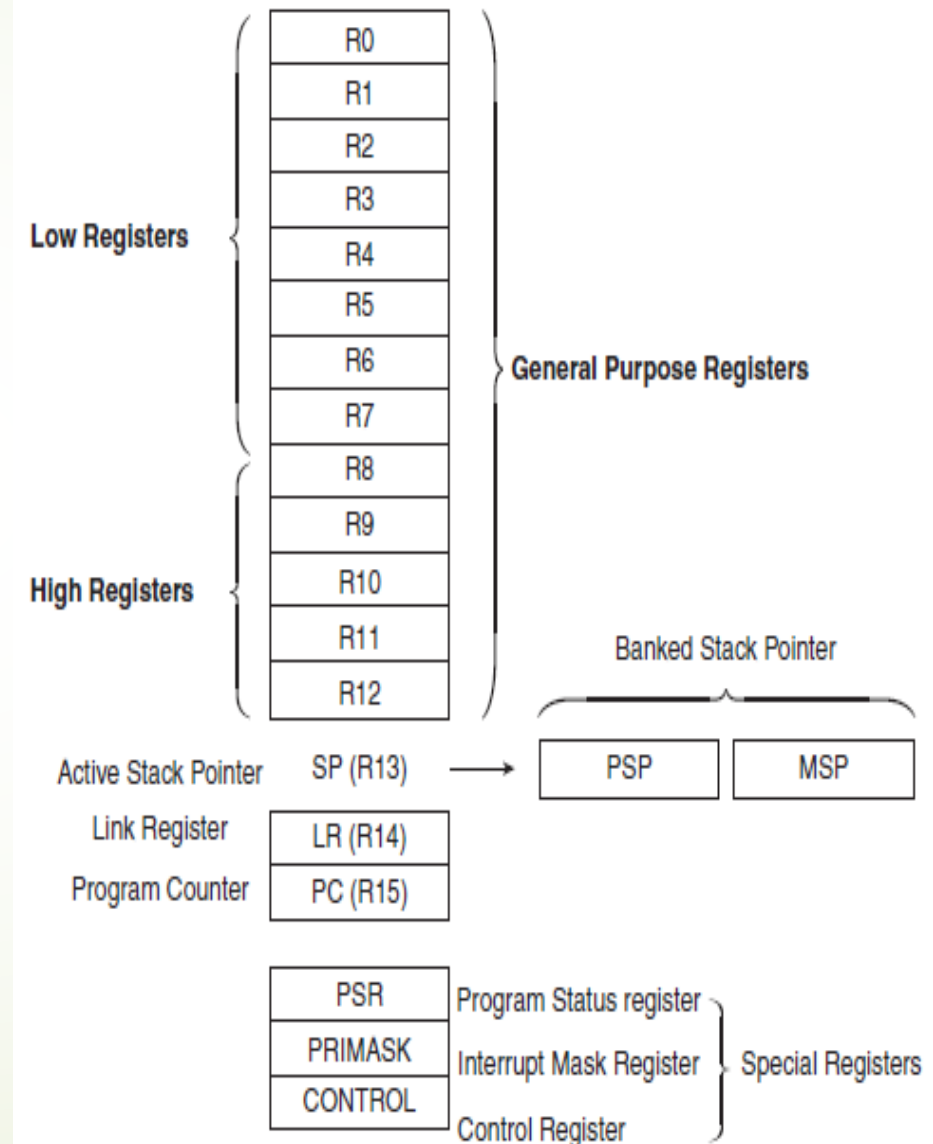


Figure 7.6 The register structure of Cortex-M0
(Reproduced with permission from ARM Limited. Copyright © ARM Limited)

7.4.2 special registers

1. Stack pointer (R13)

- ❖ Processor uses a descending stack
- ❖ Stack pointer which is also called R13 consists of 2 entities:
 - ❖ MSP (Main Stack Pointer)
 - ❖ PSP (Processor Stack Pointer)
- ❖ MSP is a default stack pointer, it is also the stack pointer used in handing interrupt handlers
- ❖ Few applications need only one stack pointer, so MSP will be used
- ❖ System with OS takes use of MSP for OS tasks and PSP for the use of applications

7.4.2 special registers

2. Link register (R14)

- ❖ As soon as function is called or an interrupt occurs, control branches to a new location
- ❖ Address to which to return to, is the current value of PC, is saved
- ❖ Register in which the return address is stored is the Link Register
- ❖ At the end of function, the contents of the Link Register can be copied back to PC
- ❖ Such a register increases speed because when functions are called, the return address is available in a processor register, rather to a stack.
- ❖ Advantage is because registers are within CPU, but memory is outside it and memory access is slower.

7.4.2 special registers

3. Program counter (R15)

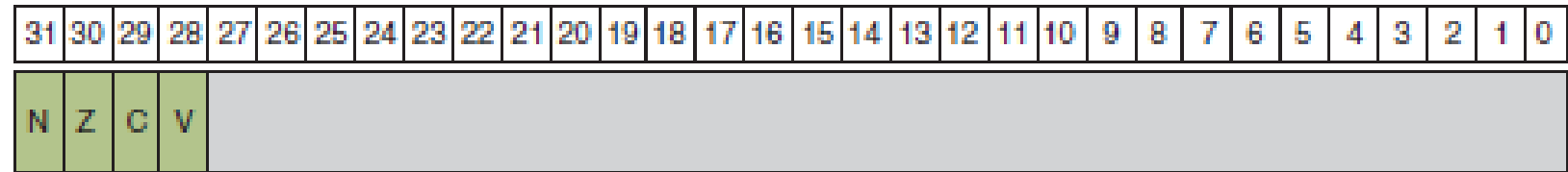
- ❖ This register sequences the flow of instruction execution
- ❖ It always contains the address of the next instruction to be executed
- ❖ It is a register which is a writable as well
- ❖ Means there are ways by which can cause branching by simply writing an address into the program counter

7.4.2 special registers

4. Program status register (PSR)

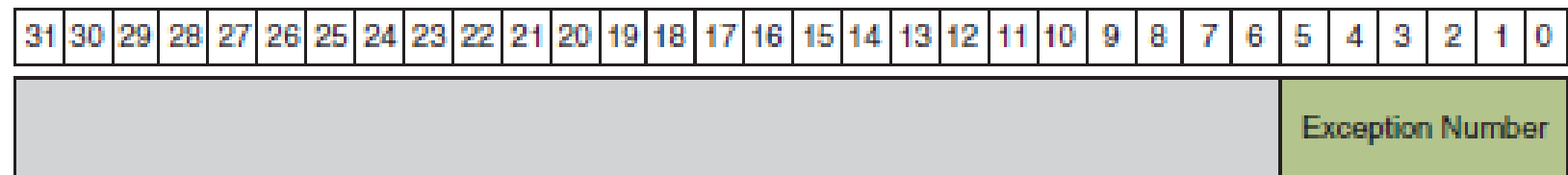
- ❖ This register holds various kinds of status information
- ❖ It is a 32-bit register which holds the status information in different bit fields, and combines three registers which are:
 - Application Program Status Register (APSR)
 - Interrupt Program Status Register (IPSR)
 - Execution Program Status Register (EPSR)

- APSR – Application Program Status Register

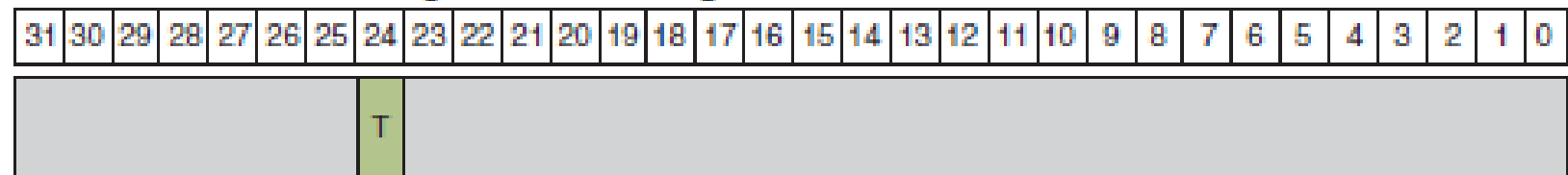


- Contains the Negative, Zero, Carry and Overflow flags from the ALU

- IPSR – Interrupt Program Status Register



- EPSR – Execution Program Status Register



- Thumb code is executed

Figure 7.7 The three registers of the PSR

- Composite register of APSR, IPSR and EPSR

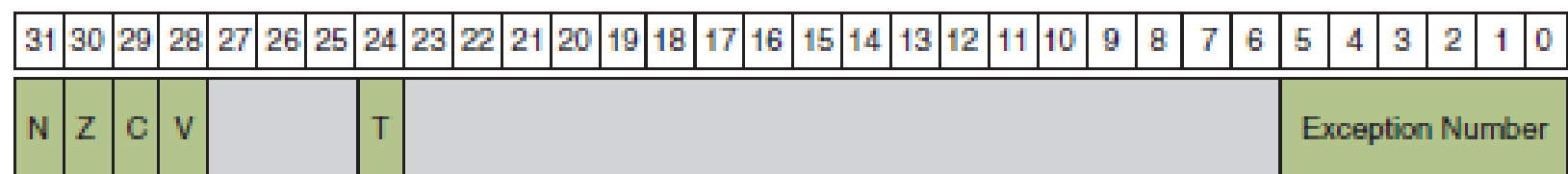


Figure 7.8 Combined PSR

- 
- 
- ❖ The status flag N, Z, C, V are in the upper four bits of APSR
 - ❖ The lower six bits of the IPSR are for the exception/interrupt number
 - ❖ Bit field 24 of the EPSR indicated the 'state' of the processor (Either in ARM or Thumb)
 - ❖ Since the processor operates only in the Thumb mode, this bit is always 1
 - ❖ Clearing this bit causes an exception

Table 7.2 Three other combinations of status registers

Register	Type	Combination
IEPSR	Read Only	EPSR and IPSR
IAPSR	Read/Write	APSR and IPSR
EAPSR	Read/Write	APSR and EPSR

Above table indicates that there are three other combinations of these status registers, that is, with two of them taken together.

7.4.3 Other Special registers

PRI (Priority Mask Register) Masks

- ❖ Certain interrupts which are 'non-maskable' and other whose priorities are configurable
- ❖ This register is related to the 'masking' of interrupts, it is bit prevent the activation of all configurable interrupts
- ❖ This register has 32 bits, out of which only the LSB is used
- ❖ Setting the LSB to '1' makes inactive
- ❖ all Interrupts with configurable priority

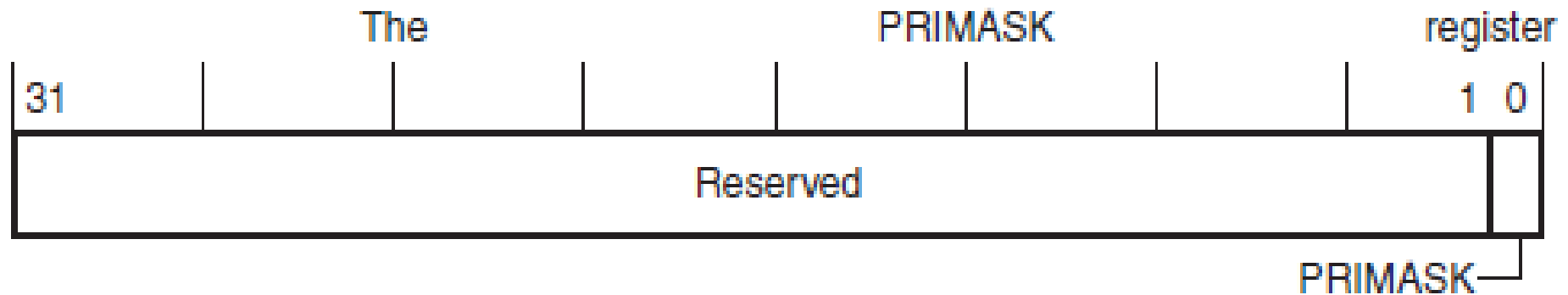


Table 2-7 PRIMASK register bit assignments

Bits	Name	Function
[31:1]	-	Reserved
[0]	PRIMASK	0 = no effect 1 = prevents the activation of all exceptions with configurable priority.

Figure 7.9 PRIMASK register and bit assignments (*Reproduced with permission from ARM Limited. Copyright © ARM Limited*)

Control Register

- ❖ This is another 32-bit register of which only one bit is meaningful
- ❖ Bit 1 selects the stack to be used when the processor is in the 'Thread' mode
- ❖ If the Bit=0, the main stack is specified to be active, otherwise the PSP is considered to be active

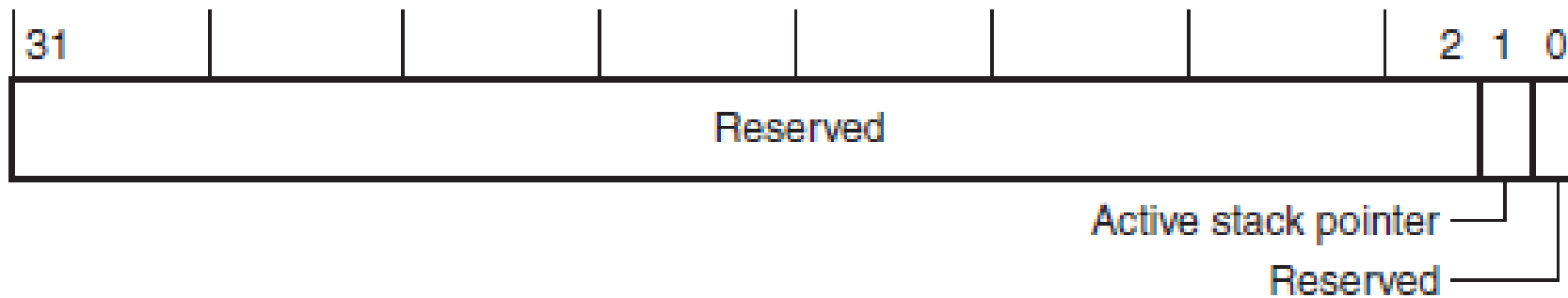


Table 2–8 CONTROL register bit assignments

Bits	Name	Function
[31:2]	-	Reserved
[1]	Active stack pointer	Defines the current stack: 0 = MSP is the current stack pointer 1 = PSP is the current stack pointer. In Handler mode this bit reads as zero and ignores writes.
[0]	-	Reserved.

Figure 7.10 The Control Register and its bit assignments (*Reproduced with permission from ARM Limited. Copyright © ARM Limited*)

7.5 Memory Model

- ❖ As address bus is of 32 bits, the addressing range of the processor is $2^{32} = 4 \text{ GB}$
- ❖ Processor uses memory mapped I/O
- ❖ 4 GB space is partitioned into space for memory (RAM and ROM) as well as for peripherals.

Core Memory Mapped Register Space,
i.e. NVIC (XN)

External Peripherals (XN)

External Memory

External Peripherals (XN)

Data Memory (code can also be placed here)

Executable region for program code and data
(vector table is fixed at address 0x0000 0000)

Device	511MB	0xE010 0000
Private Peripheral Bus		0xE000 0000
External Device	1GB	0xA000 0000
External RAM	1GB	0x6000 0000
Peripheral	500MB	0x4000 0000
SRAM	500MB	0x2000 0000
Code	500MB	0x0000 0000

XN – execute never

Figure 7.11 The memory map of Cortex-M0

Memory model of cortex-M0

1. Code:

- This part of memory is of 512 MB
- It is the lowest end of the map and is meant to store program code, even though data is allowed
- This part is realised using ROM, usually FLASH ROM
- Interrupt vector table is stored here.

Memory model of cortex-M0

2.SRAM: This part of memory is of 512 MB

Can store data, though program code is allowed here

This is part where Stack and Heap memories are initialized to be present

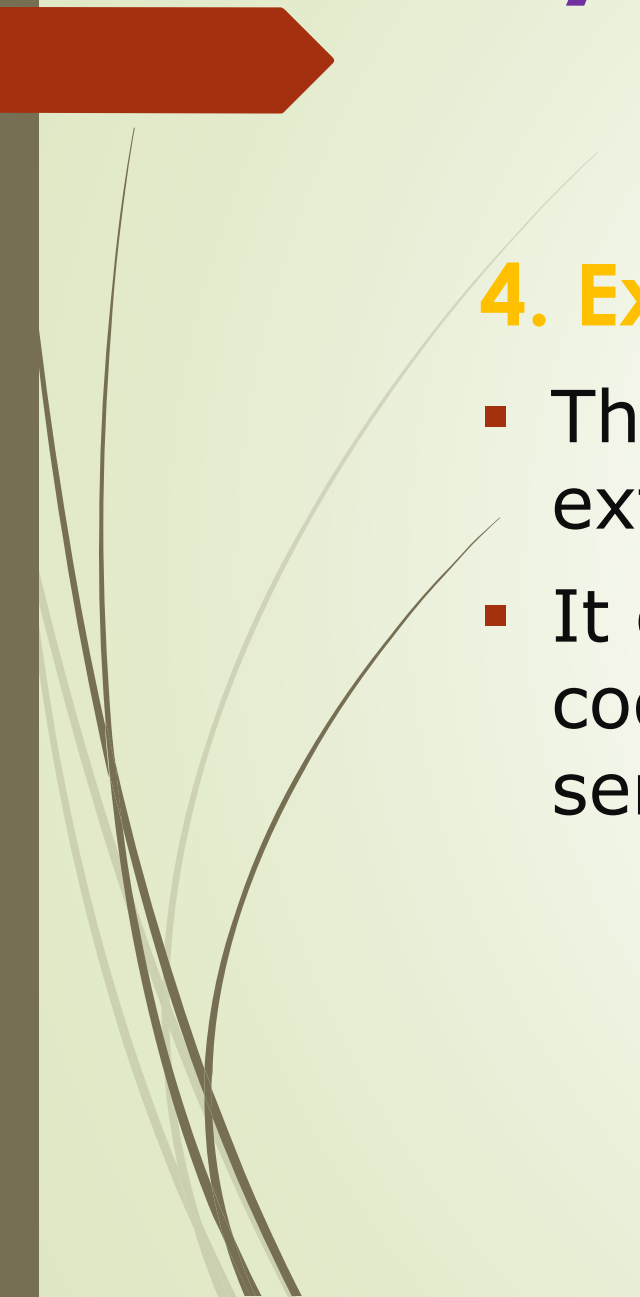
Many designs may realise this memory part using SDRAM

In programs, this section is designated as Read/Write memory

Memory model of cortex-M0

- 3. Peripheral:** Memory addresses in this region are allocated to peripheral address. Though part of it can be realised by memory device to store data. It can be used only for 'non-executable' information
- This attribute is referred to as 'EXECUTE NEVER' (XN)

Memory model of cortex-M0



4. External RAM:

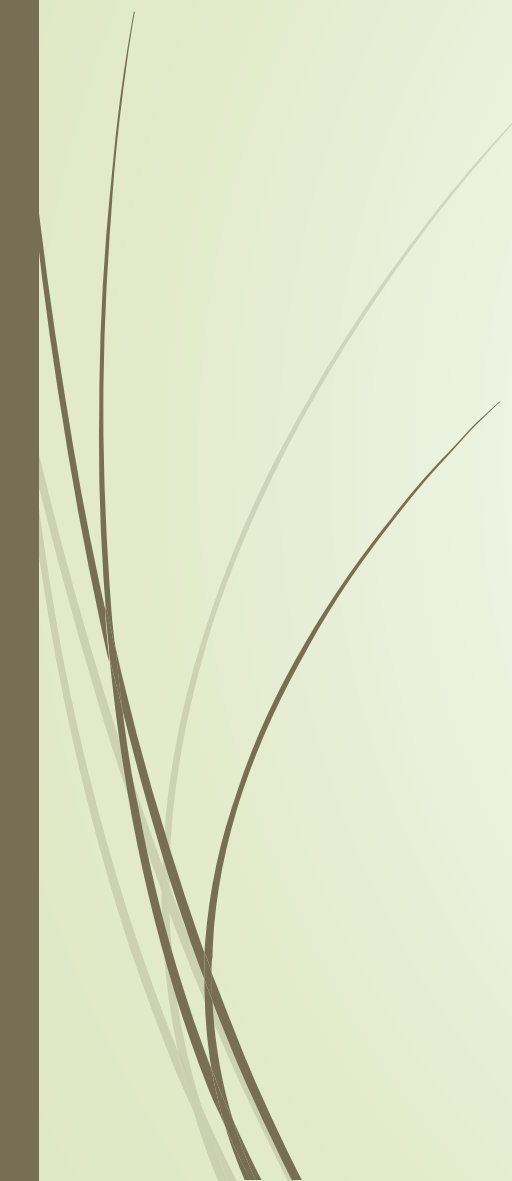
- This is 1 Gb of space which can be mapped to external RAM
- It can store data as well as executable program code and can be realised by different kinds of semiconductor memories

Memory model of cortex-M0

5. Device Memory: This is 1 Gb of space

It is a non-executable region and is used for device addresses

Memory model of cortex-M0



6. Private Peripheral Bus:

- This is a non-executable region used for addresses of the registers of the NVIC, System Tier, and System Control Block
- Only word accesses can be used in this region

Memory model of cortex-M0

7. Device Memory: This is the uppermost 512 MB constitute another non-executable (XN) device memory.

Chip designers are allowed to use it in the way they want, or just leave it as 'reserved'

7.5.1 Types and Attributes of memory

- ❖ Have seen the different sections of the memory space allocated for different applications
- ❖ Access to different areas of memory may be 'Shareable' if there is more than one processor/bus master in the system
- ❖ The order in which access is allowed to occur for region of memory, is defined by certain memory attributes
- ❖ they are:
 - Normal, Device and Strongly-ordered

1. Normal

- ❖ Memory used for program execution and data storage generally occurs within the 'Normal' designation.
- ❖ In such kind, the processor has the freedom to do 'Out of Order' accessing and even speculative reads
- ❖ Means that memory access need not be in the order in which a program is coded
- ❖ Examples: Preprogramed flash
 ROM, SDRAM, SRAM and DDR
memory

2. Device

- ❖ In the case of I/O's rules of access is stricter
- ❖ Out of order is not permitted

3. Strongly ordered

- ❖ This attribute is required where it is necessary to ensure strict ordering of the access relative to what occurred in program order before the access and after it
- ❖ This means that the processor preserves transaction order relative to all other transactions

Strongly-ordered memory always assumes the resource to be shareable

7.5.2 Out-of-Order Access

- ❖ As the program is coded, compiled and made ready to run, we assume that execution is in the order of the program lines.
- ❖ Program may change the order of execution
- ❖ It may also give higher efficiency and speed
- ❖ Similarly, the order in which the memory is accessed may also be changed.



Reasons behind it is listed as:

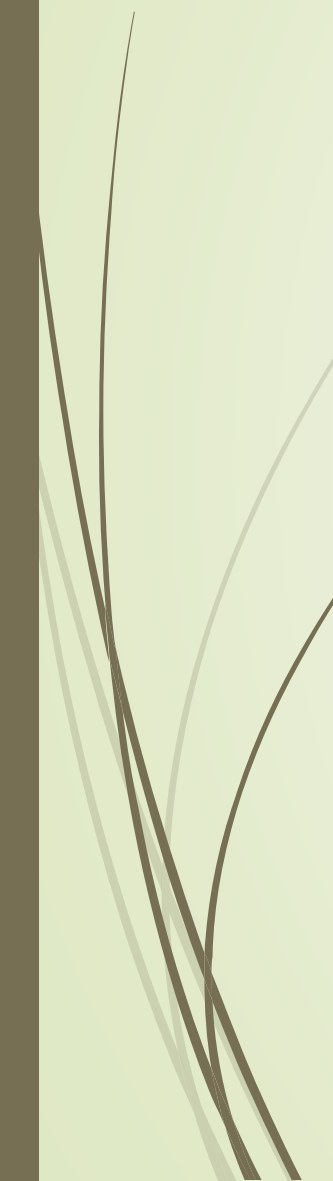
1. The processor can reorder same memory
2. Memory or devices in the memory map may have different wait states
3. Some memory accesses are buffered or speculative

7.6 Nested Vectored Interrupt Controller

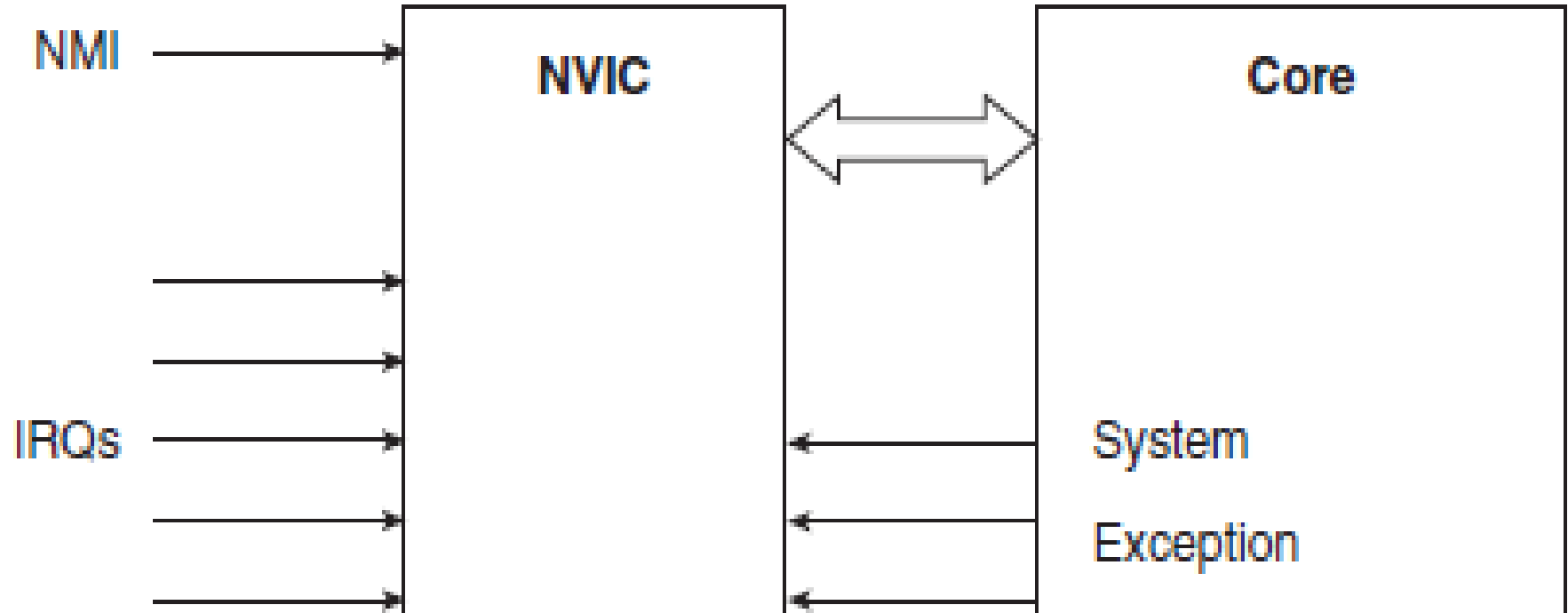
- When a currently executing program is made to stop and a new program is taken up for execution, we say that an interrupt has occurred.
- The word 'exception' is also associated with this event.
- we will use the term interrupt for all classes of such events.
- The word 'exception' will then refer to a sub class of interrupts.
- situations under which the currently running program has to be temporarily suspended to give way to another sequence of instructions.

7.6 Nested Vectored Interrupt Controller

- One case is when a peripheral needs service from the processor.
- The request from the peripheral may appear at any time with no relation to the clock of the processor.
- Such requests are 'asynchronous', that is, they appear at arbitrary instances and are not timed with the system clock.
- Such asynchronous requests (for service) from peripherals are titled as IRQs (Interrupt Request).
- There are interrupts which are generated by conditions caused by the processor — faults or certain special conditions.

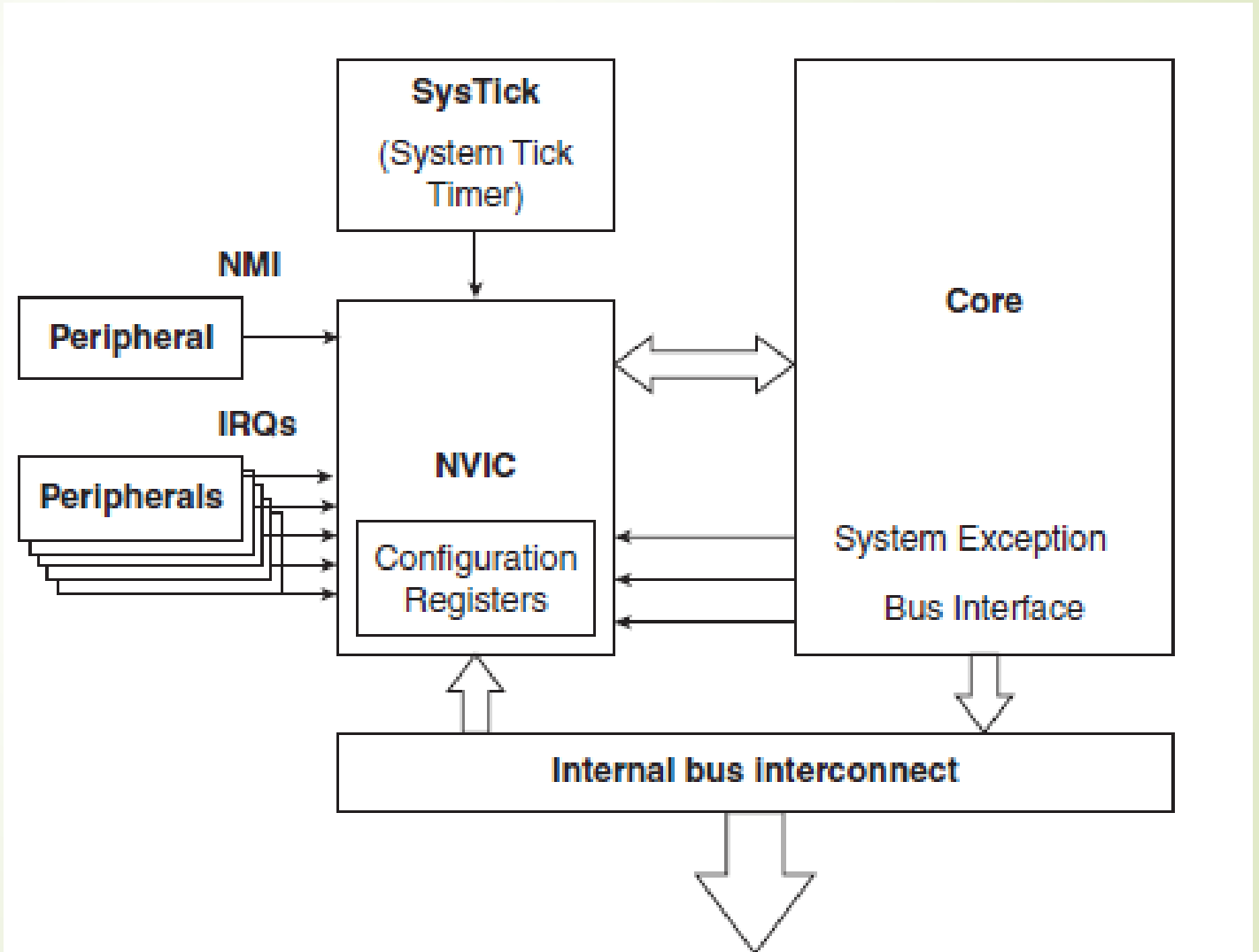
- 
- 
- Shall call them 'system exceptions' or just exceptions.
 - Some of these exceptions are 'synchronous' with respect to the CPU clock.
 - They are produced by the CPU control unit when faults or certain special conditions occur.
 - The control unit generates them only when the current instruction is terminated/completed.
 - For our Cortex-MO, 'SvCall' and 'hard fault' are synchronous exceptions
 - whereas all other exceptions are asynchronous.

- Figure illustrates the 'interrupt/ exception model' of the processor.
- Let us now move onto understanding this model in greater depth.



- 
- The NVIC in the Cortex-MO processor family provides very flexible and easy interrupt management.
 - It has features of programmable priority levels, automatic nested interrupt support, along with support for multiple interrupt masking.
 - In addition to interrupt requests from peripherals, the NVIC supports internal interrupts, for example, an input from the 24-bit timer call SysTick, which is often used by the OS.
 - There are also additional system exceptions to support OS operations, and a non-Maskable Interrupt (NMI) input.

- Figure shows the NVIC in greater detail
- Let us first talk about the SysTimer which is shown as a block in Fig.
- The SysTimer is an optional block for Cortex-M0



What are the uses of the

SysTimer?

- It operates just like an ordinary timer.
- There is a 24-bit register named SysTick Current Value Register (SYST_CVR).
- The SysTimer is a down counter. It can be loaded with a number and when the timer is enabled, the value in this register decrements down to zero.
- This event can be signaled by a Flag being set to 1 or an interrupt being generated.
- After this the Flag can be made 0 and the count value re-loaded into the register and the same process of down counting and setting the Flag may be repeated.
- SysTimer timer can be used as a reference for many applications
ex: generating square waves, PWMs, etc.
- When there is an OS in the system, the counting down to zero can be signaled by an Interrupt and this interrupt will be generated every time the count reaches zero.
- This, then becomes a regular SysTick or a time keeper for the system.

7.6.1 Features of NVIC

- The main features of the NVIC as described in the user manual are
 1. An implementation-defined number of interrupts, in the range 1_32.
 2. A programmable priority level of 0-192 in steps of 64 for each interrupt.
 3. Level and pulse detection of interrupt signals.
 4. An external NMI

The first and second features talk about IRQs and say it is 'implementation defined'.

This means that the number of interrupt (IRQs) inputs and the number of priority levels are to be decided by the application requirements, and defined by the licensees who do the final chip design.

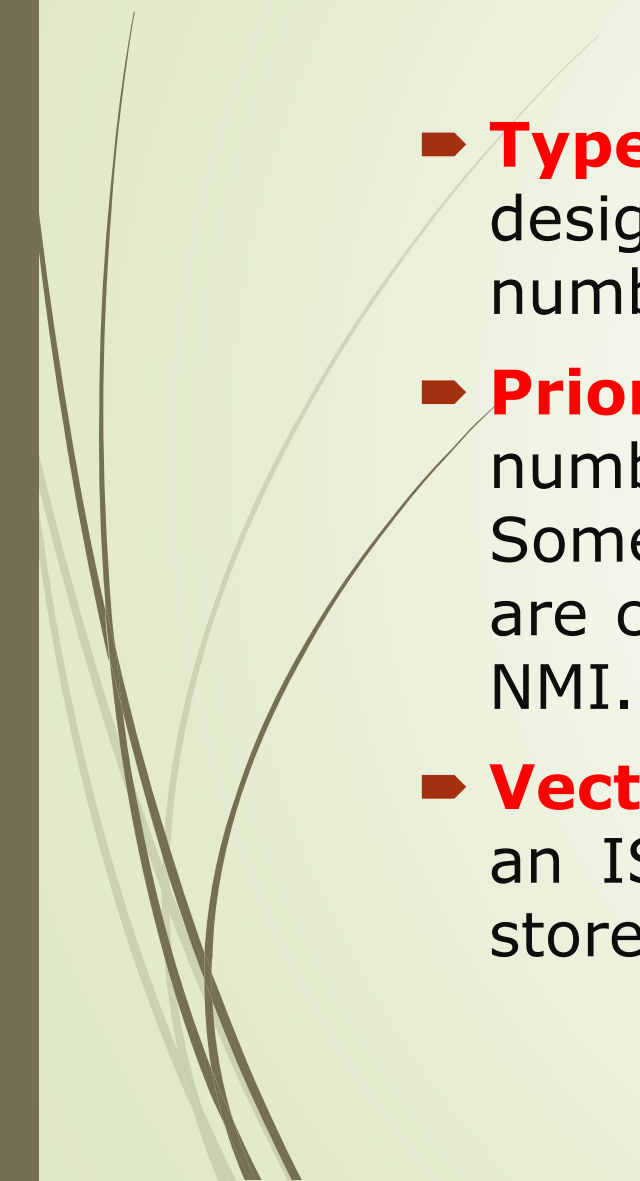
The third point is about the ways in which a device may cause an interrupt to be triggered.

The last point specifies that there is a non-maskable interrupt, which can be used by an external device.

With this basic understanding let us attempt to make a thorough study

7.6.1.1 Interrupt processing

- Cortex-M0 has an elaborate setup for handling interrupts.
- The program to which control branches to, in the event of an interrupt, is generally called an ISR (Interrupt Service Routine) or Interrupt Handler.
- Associated with each interrupt are three attributes:
 - 1.Type number
 - 2.Priority number
 - 3.Vector

- 
- 
- **Type Number:** Each interrupt has a type number (also designated as exception number) which is fixed. The type numbers are used as an index to access the vector table.
 - **Priority:** The priority of each interrupt is indicated by a number - a smaller number corresponds to a higher priority. Some specific interrupts are given negative numbers. Priorities are configurable for all interrupts except Reset, HardFault and NMI.
 - **Vector:** The term vector means 'address'. Each interrupt has an ISR. The address at which the ISR for each interrupt is stored is its 'vector'

7.6.2 Interrupt vector table

- When an interrupt occurs, its vector is to be known by the system.
- The vectors of all the implemented interrupts are stored in a table, sequentially in an order specified as 'type numbers'.
- Thus, the first vector in the table is of Reset, which has a type number 1.
- Each entry in the vector table is an address and hence is a 32-bit number, that is, a word.
- Each interrupt vector takes 4 bytes of space and so vectors are found at addresses 0x0, 0x04, 0x08 etc.
- This table is stored in the beginning of the memory map, that is, at location 0x00000000 onwards.
- This part of memory is implemented as ROM

Figure 7.14 The Interrupt Vector table

Word 47	Vector address of Interrupt 47
Word 2	Vector address of Interrupt 2
Word 1	Vector address of Interrupt 1
Word 0	Initial Value of Stack ← 0x00000000

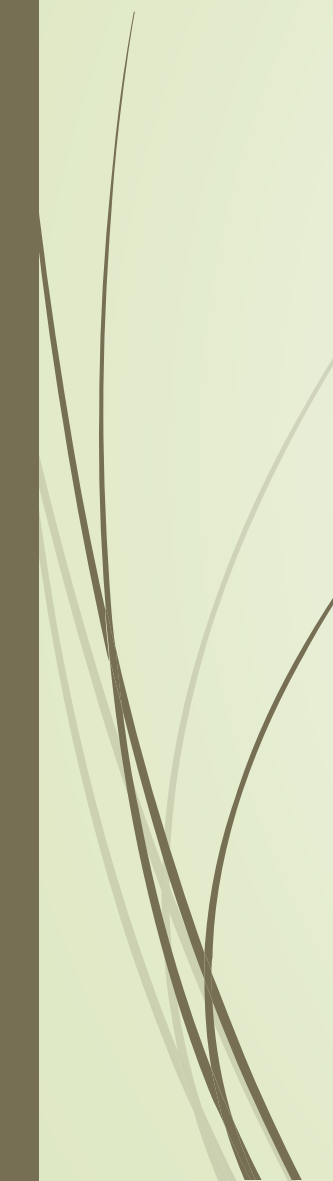
- 
- 
- The first entry in the Table is the initial value of the stack pointer (MSP).
 - The size of the vector table depends on the number of interrupts implemented in the system.
 - If there are more peripherals, there will be more interrupting devices possible, and hence a bigger vector table.
 - Cortex M allows only 15 system exception and 321RQs.
 - The vector stored as an entry in the table is the address of the interrupt handler associated with that specific interrupt.
 - For example, see the entry corresponding to Interrupt 1 which is the reset interrupt, at address 0x04
 - The entry therein is the address to which control will branch to, when the processor is powered on or when reset

Table 7.3 Type numbers and vector addresses of Cortex-M0 interrupts

Name	Interrupt type no	Vector address
Interrupts #0 – #31 (N interrupts)	16 to 16 + N	0x40 – 0xBF
SysTick (SysTick Extension)	15	0x3C
PendSV	14	0x38
SVCall	11	0x2C
HardFault	3	0x0C
Non Maskable Interrupt (NMI)	2	0x08
Reset	1	0x04



How exactly do we get the handlers of various interrupts?

- Consider the case of the reset handler.
- The system designer writes a start-up code and saves it in memory as a procedure.
- The starting address of this code is loaded into the location 0x000004 of the vector table.
- Similar is the case for the NMI handler, Hard Fault handler etc.

7.6.3 Interrupts of Cortex-M0

➤ Let us look into each of the interrupts in detail

1. **Reset:** This is a special interrupt. When the processor is powered up, control goes to the reset vector. This means that the first instruction to be executed will be taken from this address. Reset, if done any time during the course of the working of the processor, will again take program control back to the reset vector. It has the type number 1 and the highest priority level in the system, as indicated by the priority number of 3.

2. **NMI:** This stands for Non-Maskable Interrupt'. It can be associated with a peripheral or triggered by software. In any case, it is meant for critical tasks. The NMI exception cannot be masked. It cannot be pre-empted by interrupts other than reset. Its priority number is -2



3. **Hard Fault:** This produced by the faults which occur during program running. It has a priority level of -1 and a type number of 3.

4. **SVCcall:** This is a 'Supervisor Call ' Interrupt. It is a system exception triggered by the SVC instruction. It is useful in systems with an operating system. Applications use the SVC instruction to get access to device driver functions and OS kernel functions. Its type number is 11, and its priority is configurable using the register SPRH2.

5. **PendSV:** This is also related to OS-level services. In an OS environment, if no other interrupt is in the active state, PendSV is used for context switching. Its type number is 14, and its priority is configurable using register SPRH3.



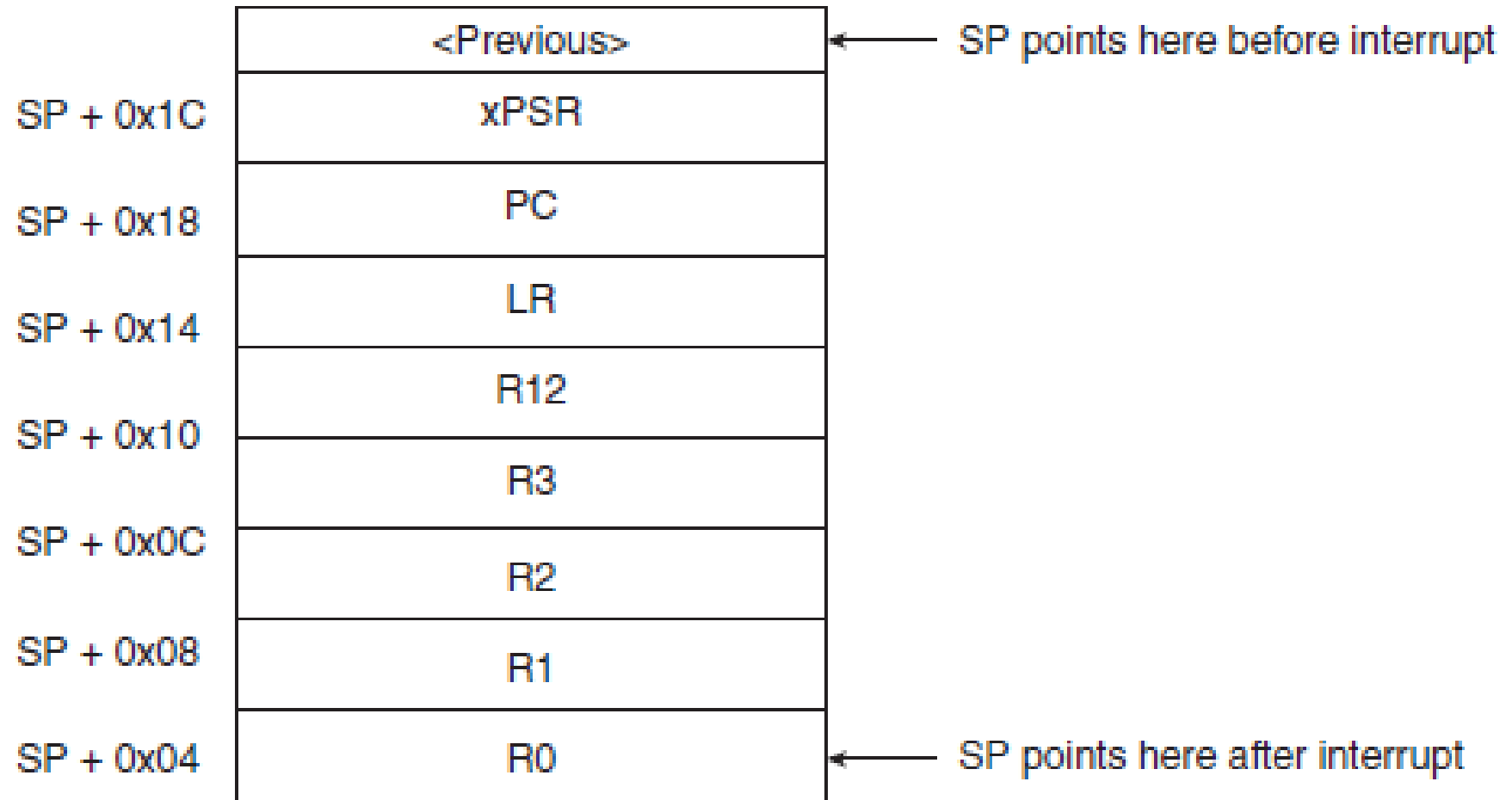
6. **SysTick**: When the system timer count reaches zero, this exception is generated. This exception can be used for general counting purposes and also as a system tick in an OS environment. It has a type number and its priority is configurable using the SPRH3 register.

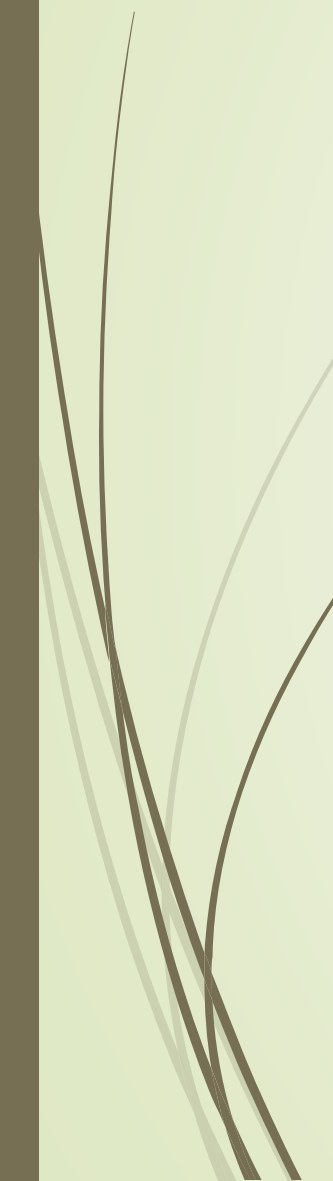
7. **IRQs**: These are the interrupts generated by peripherals, but they can also be initiated by software requests. Peripherals always communicate with the system through IRQs. The number of IRQs implemented is dependent on the chip designer.

7.6.4 Interrupt response

- At any time, each interrupt will be in one of the following states:
 1. **Inactive:** It does not need service currently.
 2. **Pending:** It needs service and is waiting to be accepted.
 3. **Active:** This is the state of the interrupt which is currently being serviced.
 4. **Active and pending:** The interrupt is being serviced by the processor the same and there is a pending interrupt from the same source.
- In all processors, interrupt handling is more or less similar
- Once an interrupt is acknowledged, the most important thing to do is context saving.
- The context is the content of the currently used registers.
- The contents of these registers are pushed on to the stack.
- If the processor is in the Thread mode, the stack that is being used could be the main stack or the process stack.
- If the processor is in the Handler mode, it is the main stack that is

Figure 7.15 Stack frame of Cortex-M0



- 
- 
- Figure shows, eight words are saved on the stack.
 - These eight words are the contents of registers R0 to R3, R12, LR, PC and PSR
 - This action of saving the context is called stacking and the saved content is called a stack frame.
 - Note that the stack frame includes the return address, which is the address of the next instruction in the interrupted program
 - This address would have been saved in the Link Register (LR) when the interrupt occurred.

7.6.5 Reset behavior

- Reset is a special case of interrupt, because it corresponds to the starting of the processor activity.
- See the sequence of actions that follow a reset (interrupt of type 1). Let's review the steps that follow this interrupt.
 1. The processor is reset (say, a Push Button reset)
 2. The 32-bit number in address 0x0 is loaded in the MSP.
 3. The reset vector is taken from address 0x04.
 4. The reset code is executed in Thread mode.
 5. The application program is accessed after this.

7.6.6 Issues in interrupt processing

What if multiple interrupts have the same priority number?

- In this case, the pending interrupt with the lowest type number (essentially those that are more important) will be allowed precedence.

Can an interrupt pre-empt a current handler?

- The word 'pre-empt' means 'remove and replace'.
- The answer is that an interrupt with a higher priority than the currently active one may be allowed to do that.
- This has to be decided as part of the policy decided for the processor and/or the operating system.

What is meant by a 'nesting of interrupts'?

- When an ISR is running, another interrupt can occur.
- If all the interrupts are equal as far as importance is concerned, the second one can interrupt the first.
- Third one can interrupt the second one and so on.
- This is called 'nesting of interrupts'. This makes it a set of 'nested' interrupts

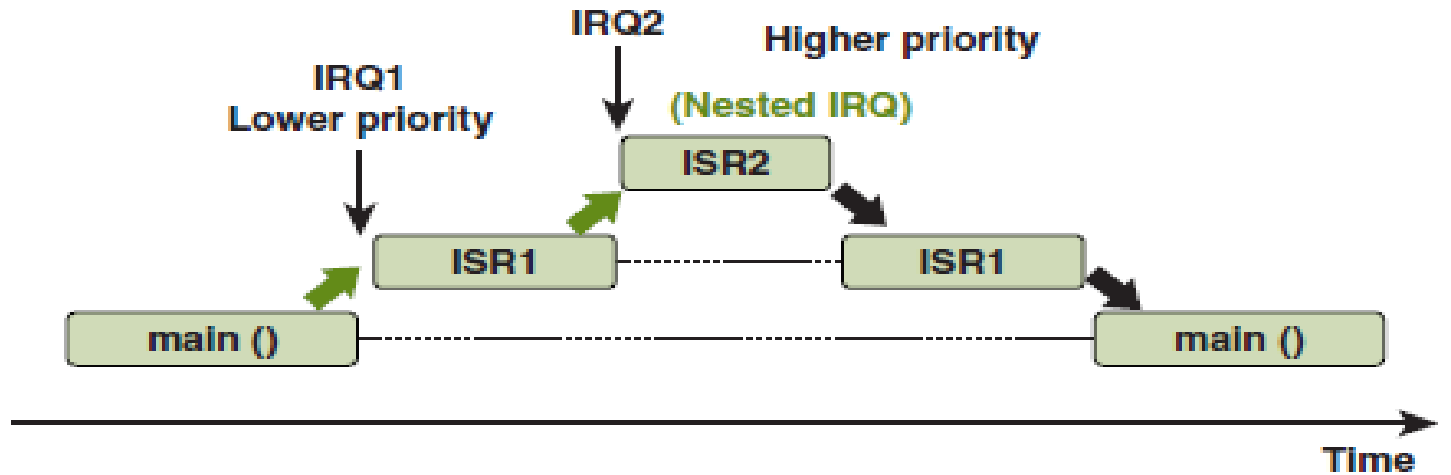


Figure 7.17 Interrupt nesting

How does the NVIC take care of interrupt nesting?

- The NVIC of Cortex-M has the capability to handle nested interrupts in a way such that the interests of the system are safeguarded.
- The points taken care of are
 1. High-priority interrupts are serviced first and they cannot be interrupted by low-priority interrupts.
 2. A high-priority interrupt may be allowed to pre-empt a low-priority interrupt handler.
 3. If an ISR is not to be interrupted by any other interrupt, all other interrupts may be masked (except the NMI and Reset of course). This can be done by setting a bit in the PRIMASK register
 4. The NVIC supports dynamic re-prioritization of interrupts. This means that priority levels can be changed by software during run time. When an ISR is being run, if all other



7.7 POWER MANAGEMENT USING SLEEP MODES

- In general, any processor design takes care of power management in various ways.
- One of the ways is by sleep modes in which minimum power is used.
- When the processor sleeps, it does not do processing and can even go to a soft OFF.
- Many embedded systems (like a mobile phone, for instance) need not work continuously, so a lot of power can be saved by putting it to sleep and then waking it up when needed.

- 
- Cortex-M0 has two inbuilt sleep modes as listed below
 1. A sleep mode that stops the processor clock. This is the normal sleep mode.
 2. A deep sleep mode that stops the system clock and switches off the PLL and flash memory
 - As a chip designer, he can use either one of them OR both of them (for different levels of sleep) according to the needs of the applications.

7.7.1 Entering Sleep Mode

- Entering the sleep mode is done by executing specific instructions.
- One instruction is the WFI (Wait for Interrupt) & other is WFE (Wait for Event)
- In the case of WFI, as executed, the processor stops program execution and goes to the sleep state.
- In the case of the execution of the WFE instruction, it checks the status of one-bit event register and acts accordingly as shown below for the bit value of 0 or 1.
- if 0, The processor stops executing instructions and enters sleep mode.
- if 1, the processor sets the registers to zero and continues executing instructions without entering sleep mode.



Why is this so?

- The event register being found to be '1' indicates that there is an external event which has to be taken care of and so 'Sleeping; is not the right thing at this point of time. Otherwise, it is okay to go to sleep.

Sleep - on - exit

- There may be embedded systems that need to work only when an interrupt comes.
- To ensure that such systems go back to sleep immediately after exiting from an ISR, there is a condition named. SLEEPONEXIT.
- This is a bit in the SCR register and if it is set to '1', it ensures that after completing an ISR, sleep state is automatically entered



How and When does the processor exit the sleep mode?

- The possible events are :
 1. Reset
 2. A debug event, if debug is enabled
 3. An exception or Interrupt with sufficient priority